Gráfica Multimedia

Producción

Lourdes Marco · lmarco@esdmadrid.es

ESDM

JavaScript

Manejando strings

Strings

Como ya hemos estado viendo, las strings o cadenas de texto son un tipo de valor primitivo en JS.
Para crear strings en JS, se pueden usar las **comillas simples** o las **comillas dobles**.

ES6 introdujo las plantillas literales (interpolación) que permiten definir una cadena de caracteres **dinámica**. Para ello usamos comillas **invertidas (`)** y **${variable}** para interpolar las variables.

JS

```
let saludo1 = “Hola”;
let saludo2 = ‘¿qué tal?’;

let nombre = ‘Pepa’;
let saludo3 = `Hola ${nombre}`;
```

Escapar caracteres

Si usamos las plantillas literales rara vez lo vamos a necesitar, pero si usamos las dobles comillas es muy probable que necesitemos escapar caracteres.

Para escapar caracteres especiales debemos usar el carácter barra invertida (\)

Por ejemplo, con las dobles comillas no podemos crear un salto de línea si no es añadiendo en la cadena de texto \r\n o \n

JS

```
let str = `Las plantillas
literales permiten
multilínea`;

let str2 = “Las comillas dobles \n no”;
```

Longitud de una cadena

Podemos saber cual es la longitud de una cadena de texto usando la propiedad **length,** la cual **devuelve un número entero** con el **total de caracteres** que contiene la cadena.

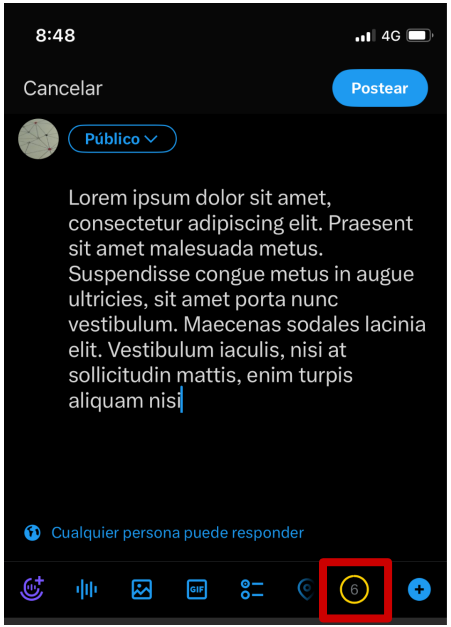
JS

```
let str = `Las plantillas
literales permiten
multilínea`;

console.log(str.length);
```

Ejemplo de número de caracteres en una GUI

Un claro ejemplo de dónde aplicar la cantidad de caracteres que hay en un texto pueden ser los post de aplicaciones como X (antes twitter). Como puede verse en la imagen de la derecha, por cada nuevo carácter introducido disminuye la cuenta atrás.



Red social X

Convertir valores en strings

Es muy común que se dé la necesidad de convertir a string un valor que no lo es. Existen diferentes formas de hacerlo:

- String(variable)
- variable.toString()
- **`texto ${variable}`**
- “texto ” + variable

La última forma es la menos recomendable, ya que no aseguras que el valor se convierte siempre.

La más recomendada son las plantillas literales.

JS

```
let num = 24;
let str = `px`;

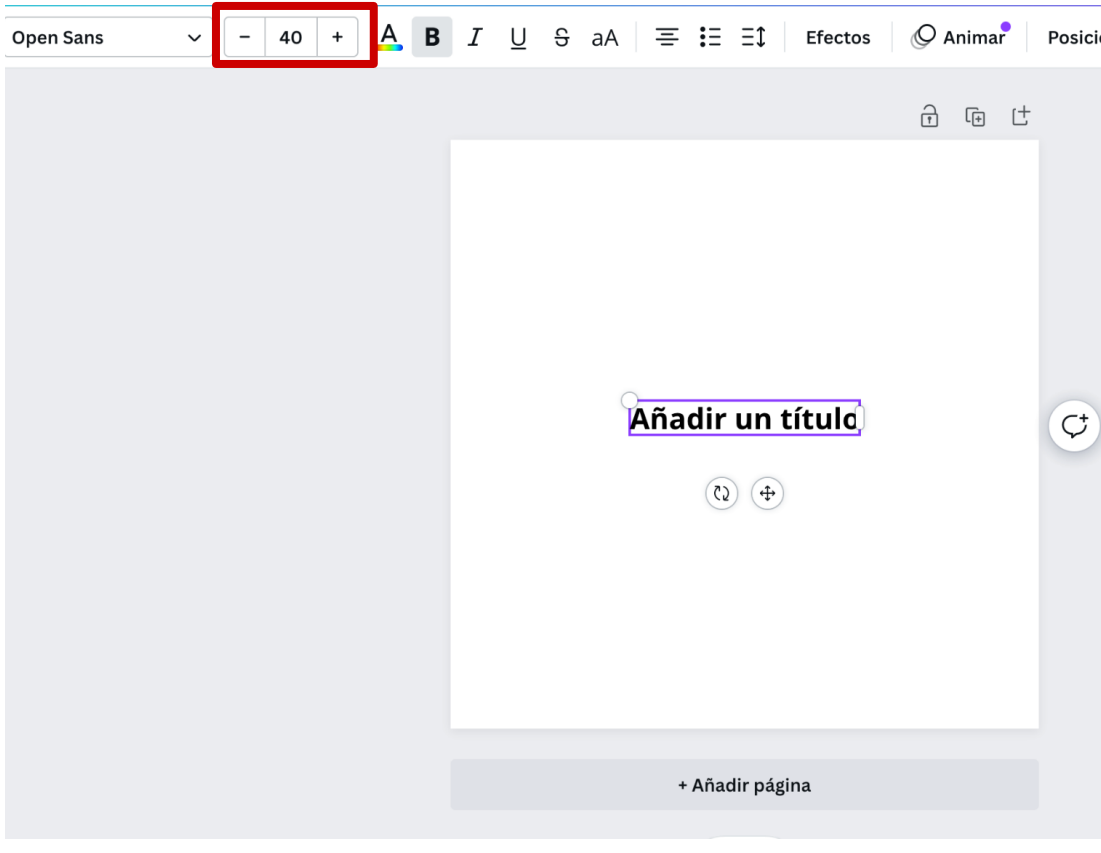
let cadena1 = String(num) + str;
let cadena2 = num.toString() + str;
let cadena3 = `${num}${str}`;
let cadena4 = num+str;

console.log(cadena1);
console.log(cadena2);
console.log(cadena3);
console.log(cadena4);
```

Ejemplo de números + string en una GUI

En interfaces como esta, donde puedo gestionar tamaños de la tipografía basados en px u otra unidad de medida, por cada clic de los botones **+** y **-** tengo que:

- Coger el valor actual (**40px**)
- Quitarle las letras -> **num = parseInt('40px', 10)**
- Sumarle o restarle uno **-> num = num++;**
- Aplicar el nuevo valor a los estilos **convirtiéndolo a string y sumando 'px'**
    - **str = `${num}px`;**



Comparar cadenas

Para comparar dos cadenas, se pueden utilizan los operadores **>, >=, <, <=** y **==.**

Estos operadores comparan cadenas basándose en los valores numéricos de los caracteres de JS.

Cuidado aquí, porque JS puede devolver un orden de cadenas diferente al utilizado en los diccionarios.

JS

```
let resultado = ‘a’ < ‘b’;
console.log(resultado); //true

let resultado2 = ‘a’ < ‘B’;
console.log(resultado2); //false
```

Comparar cadenas con localCompare()

Podemos usar el método **localCompare()** para comparar dos cadenas.
Este método devuelve -1, 0 o **1** dependiendo del resultado de la comparación por orden alfabético.

- Si la **primera cadena** es **anterior a la segunda**, devuelve -1.
- Si la **primera cadena** es **posterior a la segunda**, devuelve 1.
- Si son **iguales**, devuelve 0.

```
str1.localeCompare(str2);
```

JS

```
console.log('A'.localeCompare('B')); // -1
console.log('C'.localeCompare('B')); // 1
console.log('B'.localeCompare('B')); // 0
```

Encontrar subcadenas

Para encontrar una cadena en un texto podemos usar el método **indexOf()**

string.indexOf(substring,[fromIndex]);

Este método acepta dos argumentos:

- **substring**. La cadena a localizar
- **fromIndex**. Desde donde empezamos a contar (índice)

El método devuelve el índice de la primera aparición de la subcadena. **Si no se encuentra** la subcadena, el método **devuelve -1.**

JS

```
let str = `JavaScript mola`;

console.log(str.indexOf("mola")); //devuelve 11
```

Extraer un solo carácter

Para acceder a un carácter individual en una cadena, se utiliza la **notación de corchetes con un índice** numérico **[num]**

El índice de una cadena de texto empieza en cero, no en uno. Como puede comprobarse en el ejemplo para extraer la primera letra debemos usar **str[0]**

JS

```
let str = `JavaScript mola`;

console.log(`${str[0]}${str[4]}`);
```

Extraer subcadenas (substring)

Entendemos subcadena como un **fragmento de texto dentro de una cadena**.
Para extraer una subcadena de una cadena, se utiliza el **método substring()**

substring(startIndex,[length]);

Este método acepta dos argumentos:

- **startIndex**. El nº de índice en el que empezamos a contar caracteres.
- **length.** El número de caracteres que vamos a extraer.

JS

```
let str = `JavaScript mola`;

console.log(str.substring(0, 10));
console.log(str.substring(11, 4));

console.log(str.substring(4));
```

Si solo le pasamos el primer argumento (startindex) a la función, nos devuelve toda la cadena desde ese index.

Extraer subcadenas (slice)

ESDM

También podemos extraer subcadenas con el **método slice()**
La diferencia respecto a substring() es que a slice() le pasamos el índice de inicio y el índice de fin.

slice(startIndex, [endIndex])

JS

```
let str = `user@ejemplo.com`;
let user = str.slice(0,str.indexOf('@'));

console.log(user);
```

Eliminar espacios en blanco

Para eliminar todos los espacios en blanco iniciales y finales de una cadena, se utilizan los métodos **trimStart()** y **trimEnd()**

JS

```
let str = ` JavaScript mola `;

let cleanStart = str.trimStart();
let cleanEnd = str.trimEnd();

console.log(cleanStart);
console.log(cleanEnd);
```

Cambiar a mayúsculas o a minúsculas

Podemos cambiar una cadena de textos a mayúsculas o a minúsculas según nos interese.

- Para convertir todo a mayúsculas usamos el método **toUpperCase()**
- Para convertir todo a minúsculas usamos el método **toLowerCase()**

JS

```
let str = `JavaScript mola`;

console.log(str.toLowerCase());
console.log(str.toUpperCase());
```

Rellenar cadenas

Podemos rellenar una cadena por el principio o al final. Por ejemplo si queremos mostrar solo las primeras letras de un dato y el resto poner asteriscos. O si queremos completar un número con 0 por delante.

- **Para rellenar el principio** de una cadena usamos el método **padStart()**
- **Para rellenar el final** de una cadena usamos el método **padEnd()**

Ambos métodos tienen dos parámetros: **caracteres totales** de la cadena y **con qué la rellenamos**.

JS

```
let str = '1234';
let completa = str.padStart(8,'0');
console.log(completa);


let str2 = 'abc';
let completa2 = str2.padEnd(6,'*');;
console.log(completa2);
```

Comprobar si una cadena contiene otra

Con el método **includes()** podemos comprobar si una cadena contiene otra. Por ejemplo, imaginemos que queremos comprobar si el email introducido por el usuario incluye la @.

Si la cadena contiene la cadena que buscamos, el método **devuelve true**, de lo contrario devuelve false.

JS

```
let email = 'user@ejemplo.com';
console.log(email.includes('@'));
```